

One person, eight names.

A Graph RAG system over 190 unsealed federal court filings. Built on ArangoDB, measured earnestly, and brutally honest about where it falls short.

3,338 PASSAGES

19,009 MENTIONS

2,014 PEOPLE, PLACES, ORGS

488 SAME-PERSON LINKS

BACKGROUND · 6–8 MINUTES

My bio, why this role

SUMMARY & ENGAGEMENTS

Where you are now, in one sentence.

Two or three engagements — favour ones where you explained something technical to a non-technical decision maker.

GRAPH EXPERIENCE & WHY THIS ROLE

Which systems, at what depth, on what problems.

Why ArangoDB, why now, why solutions architecture.

The rest of this deck is one long answer to the same question: I didn't read about Graph RAG. I built one, measured it, and can tell you where it fails.

The same product, two audiences

TO AN ENGINEER

01 Split — documents into passages; size, overlap and separators are all configurable

02 Extract — the model finds people, places and organisations, and how they relate. Each becomes a node in the graph.

03 Cluster — group related nodes into topics, then group small topics into larger ones

04 Summarise — the model writes a short report on each topic

Two services: the Importer builds the graph, the Retriever answers questions against it.

TO AN EXECUTIVE

- A queryable map of who and what across every document.
- Answers with citations to the page — so a person can check them.
- Runs where your data lives, including air-gapped.
- One database instead of three to secure and staff.

The executive question is never "how does the retriever work". It's what can my team ask on Monday that they couldn't ask on Friday — and can they trust the answer.

Five retrieval methods, five different questions

Method	Answers	Speed
Global	Themes across every document, using the topic reports	medium
Local	One person or organisation, and everything linked to it	low
Deep	Multi-step research; the model plans its own steps	higher
Instant	Fast answers with references	low
Custom	Your own rules, over your own data	varies

Global Search is the one nobody else ships, and it is available through the API only. Graph search needs a name to start from. A question like "which courts appear in these documents" gives it none. My own system falls back to plain similarity search on 11 of 35 questions for exactly that reason.

Where it fits — and where it doesn't

STRONG FIT

- Relationships carry the meaning — investigations, compliance, supply chain.
- Answers span documents — two facts in two files must be joined.
- Questions about the whole set — counting and summarising, where similarity has nothing to rank.
- Data that cannot leave — regulated, privileged, classified.
- The same person or place appears under many names.

NOT THE BEST FIT

- Purely conceptual questions. There is no name to start from, so plain meaning-based search (cosine) wins.
- Small, uniform document sets. One author and one vocabulary, so the extra machinery earns nothing.
- Simple lookup. If every question is "find the passage about X", a plain vector index is the answer.

The Importer has a setting, `rag_mode`: "vector_rag", that skips entity extraction altogether. The product already agrees with this column.

What a prospect is actually weighing

Alternative	Its appeal	Where it stops helping
Vector DB alone	Simplest thing that works	No structure. It cannot answer "both X and Y", and it cannot count.
Neo4j + LangChain	Mature engine, large hiring pool	You assemble the RAG layer yourself. Vectors live in a second database, and the join between the two is where bugs live.
Microsoft GraphRAG	Where the idea was published	A library, not a service. You operate all of it.
Off-the-shelf frameworks	Fastest path to a demo	The demo is the easy part. Matching the same person across documents, and re-loading changed documents, are where the year goes.

What I won't claim: I haven't benchmarked ArangoDB against Neo4j or Neptune. ArangoDB is written in C++ rather than Java, so there are no garbage-collection pauses and the memory floor is lower. My 30–91 ms graph queries are consistent with that. They are not proof of it.

The same person, written eight ways

190 unsealed filings from *Giuffre v. Maxwell*. Depositions, exhibits and motions, written by opposing lawyers, across many authors and many years.

WHY THESE DOCUMENTS

The same name is written differently depending on the document: RECAREY, JOSEPH in a case caption, Det. Recarey in a transcript, Mr. Recarey in a brief.

Search one spelling and you reach only the evidence that uses that spelling.

THE COUNTER-CASE

Jane Doe, Jane Doe 2, Jane Doe 3 — near-identical to any name-matching algorithm, but three different people.

A system that merges them is worse than one that merges nothing.

Unsealed public record. Every model runs on my own machine. Nothing was sent to an outside service.

Data model

Collection	Type	Count	Purpose
je_chunks	doc	3,338	One row per passage of text
je_raw_extractions	doc	3,338	The model's raw output, kept for audit
je_entities	doc	2,014	person · organization · location · facility · court
je_mentioned_in	edge	19,009	Which name appears in which passage
je_same_as	edge	488	Two names, same person
je_possible_duplicates	edge	413	Matches the system refused to make, queued for review

EVIDENCE ATTACHES TO THE MENTION, NOT THE PERSON

A role and its supporting quote belong to one mention in one passage. Merging two names can never overwrite the trail.

LINKS ARE ADDED, NEVER MERGED AWAY

Nothing is deleted. Search follows a whole name family in one step, and a reviewer can undo any single link.

Demonstration

01 A question about a person — watch the graph path. 1,770 passages reachable, eight returned.

02 A question the documents cannot answer — it reports that it found no name to start from, then declines: "no judicial opinions, only party filings."

03 The graph itself — Joseph Recarey's seven different spellings. Then Jane Doe 2, and fifteen matches the system refused to make.

04 AQL, ArangoDB's query language — the name family, the review queue, and how much more evidence a family reaches than a single spelling.

Watch the channel tags: vector#26 lexical#7 graph#11 means three independent methods each picked the same passage. When they agree, that is the strongest signal the system produces.

What I traded, and what it cost

TWO STORES INSTEAD OF ONE

The graph sits in ArangoDB; the text and vectors sit in LanceDB. It was the quick choice, and it shows: a few passages exist in the graph with no matching row in LanceDB.

NO PERSON-TO-PERSON LINKS

This is an index of who appears where, not a map of how people relate. A second step would only mean "mentioned in the same passage", which here says very little.

SAME-PERSON LINKS FOR PEOPLE ONLY

The rule that works for people runs backwards for places. A trial run produced 11 wrong links out of 14. I stopped it and limited the pass to people.

RATHER MISS A LINK THAN INVENT ONE

413 refusals are queued for review. A wrong merge invents a connection between real people. A missed one only means we find less.

What I'd change to run this at scale

First, I'd delete LanceDB. ArangoDB 3.12 has a native vector index. Two stores collapse into one, an entire class of consistency bug disappears, and a join across two systems becomes a single lookup.

CLUSTER AND SHAPE

- SmartGraphs with partition_id, so graph queries stay on a single shard.
- Satellite collections keep the entity index on every node.
- Partition by legal matter, which is also the customer boundary.

CORRECTNESS BEFORE FEATURES

- Decide first, then write. Built and measured: it now declines when it should in 11 of 15 runs, up from 0, with no wrong refusals in 105.
- A human review step, budgeted and staffed.
- A review screen for the 413 flagged pairs.
- A second, slower model to re-order the final shortlist (a cross-encoder reranker).

Unsolved: adding new documents without rebuilding. Matching names across all 190 filings took 19 hours, and that cost grows with the number of mentions, not the number of documents. Adding one filing without redoing the whole graph is the hard problem.

Pitfalls learned along the way

Each one taught me something I now carry into the next build and related projects.

1 · A LIBRARY DEFAULT COST 9.6% OF THE GRAPH

The name-matching library (Jaro-Winkler) defaults to case-sensitive. In documents full of ALL-CAPS captions, that produced 38 separate "Mr. Barton" nodes.

→ Fixed — compare both names in the same case, and sort the candidate list by score before cutting it to five. Then re-ran the pass over every document.

2 · TWO DIFFERENT MAXWELLS MERGED INTO ONE

Ghislaine and her father Robert. Comparing two names at a time cannot tell a fuller version of one name from a different person entirely.

→ Fixed — judge the whole family of names at once instead of pair by pair, using how often each appears, how widely across documents, and in what role. Anything still uncertain is flagged, not merged.

3 · THE MODEL NEVER SAW THE TOP OF ITS OWN INSTRUCTIONS

Ollama silently cuts any prompt down to 4,096 tokens, without warning — and it cuts from the front. Mine were 4,235, so every test run lost the opening of its own system prompt, starting with "use only these sources".

→ Fixed — set the context length explicitly to 8,192 in config rather than inheriting a default, and log the prompt length so the next cut is visible.

4 · BIGGER MODEL, WORSE RESULTS

A 30-billion-parameter model of the mixture-of-experts kind scored 11/16. A plain 8-billion model scored 15/16, at the same speed.

→ Settled — kept the smaller model, on the evidence. I also abandoned a prompt change that made both models worse, rather than trying to rescue it.

Every one was found by auditing real output — not by reading code, and not by a test. Every one is fixed and re-measured.

A human stayed in the middle of every step. Claude Code wrote most of the code. Every data structure, threshold and algorithm choice was argued out before it went in. Working the problem with the model rather than delegating it to the model is where the experience compounded.

Four channels, fused by rank

Channel	Mechanism	The question that justified it
vector	Meaning-based similarity (cosine)	One passage names him, and it ranks first by similarity
density	Documents with the most passages in the shortlist	A 102-passage transcript: ranked 14th by its best passage, 2nd by density
lexical	Keyword search after the model widens the query (BM25)	The word "obiter" appears nowhere; 35 passages say "held that"
graph	Match a name, follow one link, collect its mentions	"Both Maxwell and Epstein" — a single vector cannot express "both"

RANK FUSION (RRF)

$\Sigma 1/(60 + \text{rank})$. It uses position only, so channels that score on completely different scales can be combined without anyone inventing weights.

WHY HALF THE SLOTS ARE RESERVED

Half always go to plain similarity search. Treating all four channels equally once fixed one question and broke another in the same change.

Saying "I don't know" — two bugs, pulling opposite ways

Fixing the first one caused the second. Only measurement found it.

FIRST: IT ANSWERED EVERYTHING

Asked something the documents could not answer, it described what they did contain and offered that as the answer. Four prompt rewrites failed — the prompt already forbade it.

The cause was structural: one model, in one pass, both decided whether it had enough evidence and wrote the answer. Writing something is the easier path.

→ Fixed — split the two. Step 1 decides, and its reply format has no field big enough to hold prose. Step 2 writes, and runs only if step 1 said yes.

THEN: IT REFUSED THINGS IT COULD ANSWER

"Who were the defense attorneys in this case?" was declined. The answer was sitting in the top result: "MS. BORJA: Mary Borja for Defendant, Alan Dershowitz."

Step 1 read all eight passages in one call and gave a yes/no for each. That looks independent. It is not — remove one unrelated passage and the verdict on the top result flipped to yes. Swap in a different one and it flipped back.

→ Fixed — one passage per call, each call seeing exactly one passage. The set stops being a variable.

0 / 15 REFUSED BEFORE

11 / 15 REFUSED AFTER

0 WRONG REFUSALS IN 105 RUNS

49s DOWN FROM 63S

What is measured, and what is not

MEASURED

35 questions in seven categories. The predicted winner was written down and locked before any strategy ran.

- Graph found no name to start from on 11 of 35
- Won at least one of the eight final slots on 18 of 35
- Changed the document set on 14 of 35

NOT MEASURED — DELIBERATELY

No human-marked correct answers, so none of the standard ranking scores (recall@k, nDCG) can be computed. Judging correctness requires a person, and cannot be delegated.

An answer key written by a model would measure the judge, not the systems.

The evaluation shows that the strategies behave differently. It does not show which one is right. I would rather say that than quote a number I cannot defend.

Challenges for improving the system

Each one is scoped, understood and costed. None of them is a surprise, which is the point of measuring a system rather than only building it.

- Same-person links for places and organisations. Limited to people for now; the matching rule has to change depending on the type.
- 413 flagged pairs awaiting a decision. The system knows what it is unsure about. It needs a screen that shows a reviewer.
- Correctness not yet verified by a person. Human work by design, not a gap waiting on automation.
- Answering a different question than the one asked. Separate from declining to answer, and not fixed by it.
- Adding new documents without rebuilding. The hardest, and the one that matters most at scale.